Angular Learning Series
Topics 38 – 41

Angular Router · Route Guards · Lazy Loading · Custom Directives

Generated: 17 May 2026

TOPIC 38 — Angular Router (Routes, Navigation, Parameters)

Angular Core — HTTP & Routing | Difficulty: Intermediate | Relevance: Critical

---

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — The Router API is large and stateful; understanding how URL changes drive component lifecycle (rather than the other way around) requires a mental model shift from component-centric thinking.

Angular Relevance: Critical — Every multi-page Angular app lives or dies by the Router; it controls what renders, what data loads, what guards run, and how deep links work in production.

Section 1 — Explanation

THE GPS NAVIGATION ANALOGY

Think of Angular Router as a GPS system built into your app.

- The URL is the destination address the user types in.
- The route configuration is the map — it defines every valid road and where each road leads.
- The RouterOutlet is the windshield — whatever building you arrive at gets displayed there.
- RouterLink is the 'Navigate To' button on your GPS — you tap it and the GPS computes the route.
- ActivatedRoute is the GPS's current trip data — it tells you right now what street you're on, what turn parameters you passed, and what query strings were included.
- Router.navigate() is the GPS's programmatic recalculation — your code decides the next destination without the user pressing anything.

Just as a GPS doesn't physically move the car but instructs what path to take, the Router doesn't create or destroy components directly — it matches the URL to a configuration and lets Angular's component machinery handle the rest.

TECHNICAL EXPLANATION

Angular Router is a URL-to-component mapping engine with side effects. When the URL changes (user click, back button, or code call), the Router:

- 1. Parses the new URL into a route tree
- 2. Matches segments against your route configuration
- 3. Runs any applicable Guards (Topic 39)
- 4. Resolves any Resolvers (Topic 39)
- 5. Activates the matched component(s) inside <router-outlet>
- 6. Updates ActivatedRoute with current params, query params, and data

The Router is location-aware — it integrates with the browser's History API so the back button works naturally.

QUICK REFERENCE TABLE — Router Core Concepts

Concept	What It Is	Where It Lives
Routes array	Config mapping URL patterns to components	app.routes.ts
<router-outlet>	Placeholder where matched component renders	Template
RouterLink	Directive for declarative navigation	Template
Router service	Programmatic navigation & URL queries	Injected service
ActivatedRoute	Current route's params, query params, data	Injected service
RouterLinkActive	Adds CSS class when route is active	Template
Route params (:id)	Dynamic URL segments	URL path
Query params (?q=)	Optional key-value pairs after ?	URL query string
Fragment (#section)	Anchor within the page	URL fragment
provideRouter()	Registers router in standalone app	app.config.ts

Section 2 — Connection to Prior Concepts

Directly builds on:

- Topic 22 — async Pipe: ActivatedRoute exposes params as Observables; async pipe is the cleanest way to consume them in templates.
- Topic 21 — takeUntilDestroyed & DestroyRef: Any manual subscription to ActivatedRoute.params$ or queryParams$ must be cleaned up.
- Topic 27 — Services & Dependency Injection: Router and ActivatedRoute are both injectable services.
- Topic 36 — HttpClient & API Integration: The most common Router pattern is: route param changes -> switchMap -> HTTP call.
- Topic 25 — Component Lifecycle Hooks: Router reuses component instances on param-only changes — ngOnInit does NOT re-fire.

Bridging note: Everything you learned about services (Topic 27) applies directly here — Router and ActivatedRoute are just Angular's own services tied to browser URL state. Your existing mental model of 'inject -> use' transfers completely.

Single most-direct extension of: Topic 27 — Services & Dependency Injection.

Section 3 — Code Example

Plain TS — URL pattern matching (concept in isolation):

```
const routes = [
  { path: 'home', component: 'HomeComponent' },
  { path: 'users/:id', component: 'UserDetailComponent' }, // :id is a param slot
  { path: '', redirectTo: 'home', pathMatch: 'full' },
  { path: '**', component: 'NotFoundComponent' }, // wildcard — must be last
];

// When URL = '/users/42'
// Router finds 'users/:id', extracts { id: '42' } — always a string
```

app.routes.ts — Full routing setup:

```
import { Routes } from '@angular/router';
import { HomeComponent } from './home/home.component';
import { UserListComponent } from './users/user-list.component';
import { UserDetailComponent } from './users/user-detail.component';
import { NotFoundComponent } from './not-found.component';

export const APP_ROUTES: Routes = [
  { path: '', redirectTo: 'home', pathMatch: 'full' },
  { path: 'home', component: HomeComponent },
  { path: 'users', component: UserListComponent },
  { path: 'users/:id', component: UserDetailComponent },
  { path: '**', component: NotFoundComponent }, // MUST be last
];
```

app.config.ts:

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { APP_ROUTES } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(APP_ROUTES),
  ],
};
```

app.component.ts — the shell:

```
import { Component } from '@angular/core';
import { RouterOutlet, RouterLink, RouterLinkActive } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, RouterLink, RouterLinkActive],
  template: `
    <nav>
      <a routerLink="/home" routerLinkActive="active-link">Home</a>
      <a routerLink="/users" routerLinkActive="active-link">Users</a>
    </nav>
    <router-outlet />
  `,
})
export class AppComponent {}
```

user-detail.component.ts — reading route parameters reactively:

```
export class UserDetailComponent {
  private route = inject(ActivatedRoute);
  private router = inject(Router);
  private userService = inject(UserService);

  // Reactive: whenever :id changes, re-fetch automatically
  // switchMap cancels in-flight requests when param changes
  user$ = this.route.paramMap.pipe(
    switchMap(params => {
      const id = params.get('id')!;
      return this.userService.getUser(id);
    }),
    takeUntilDestroyed(),
  );

  goBack(): void {
    this.router.navigate(['/users']);
  }
}
```

Query parameters example:

```
// Navigating WITH query params
this.router.navigate(['/users'], {
  queryParams: { page: 2, sort: 'name' }
  // Produces: /users?page=2&sort=name
});

// Reading query params reactively
this.route.queryParamMap.subscribe(params => {
  const page = params.get('page') ?? '1';
  const sort = params.get('sort') ?? 'id';
  this.loadUsers(Number(page), sort);
});
```

Section 4 — Before & After Refactor

BEFORE — Snapshot-only param reading (breaks on same-component navigation):

```
// WRONG: snapshot is a one-time read taken at component mount
ngOnInit(): void {
  const id = this.route.snapshot.paramMap.get('id')!;
  this.userService.getUser(id).subscribe(u => this.user = u);
  // If user navigates /users/1 -> /users/2, this never re-runs
}
```

AFTER — Observable param stream (reacts to every navigation):

```
// CORRECT: reactive to every :id change
user$ = this.route.paramMap.pipe(
  switchMap(params => this.userService.getUser(params.get('id')!)),
  takeUntilDestroyed(),
);
```

Why this refactor matters: Angular's Router uses component reuse for performance — it doesn't destroy and recreate the same component just because a URL param changed. Using paramMap$ (the Observable) means your data pipeline responds to every URL change regardless of component lifecycle, while snapshot only runs once at initialization.

Rule of thumb: Use snapshot only when the component is always freshly created. Use paramMap$ whenever the same component can be navigated to with different params while already active.

Section 5 — Common Mistakes & Misconceptions

Mistake 1 — Using snapshot for components that get reused

The mistake	this.route.snapshot.paramMap.get('id') in ngOnInit()
Why developers make it	It's simpler, synchronous, and works perfectly on first load — the bug only appears when navigating between records of the same type
How to avoid it	Default to this.route.paramMap.pipe(switchMap(...)) — use snapshot only when you can prove the component is always fresh
Angular consequence	Stale data displayed silently — no error, no warning, user sees wrong record

Mistake 2 — Forgetting that route params are always strings

The mistake	Passing params.get('id') directly where a number is expected
Why developers make it	URLs look like /users/42, so 42 feels like a number
How to avoid it	Always convert: Number(params.get('id')) or +params.get('id')!
Angular consequence	API calls pass the string '42' instead of the number 42 — may work on some backends, fails on strictly typed ones

Mistake 3 — Putting ** wildcard before other routes

The mistake	Placing { path: '**', component: NotFoundComponent } in the middle of the routes array
Why developers make it	Order doesn't feel significant in a declarative config object
How to avoid it	The Router matches routes top to bottom, first match wins — wildcard must always be the final entry
Angular consequence	All routes below the wildcard become permanently unreachable — they silently show NotFoundComponent instead

Section 6 — Do's and Don'ts

DO	DON'T
DO: Use paramMap$ Observable for components that handle multiple records	DON'T: Use snapshot.paramMap when the same component can receive different params while active

DO: Always Number() or + convert route params before passing to services	DON'T: Assume params are typed — they are always string | null from the Router
DO: Put { path: '**' } as the absolute last route	DON'T: Put wildcard before any real routes — it will capture them
DO: Import RouterLink, RouterLinkActive, RouterOutlet explicitly in standalone components	DON'T: Forget to import Router directives — the template compiles but links do nothing
DO: Use [routerLink]="['/users', id]" (array syntax) for dynamic segments	DON'T: Build URL strings manually — brittle and error-prone
DO: Use queryParamsHandling: 'merge' when navigating to preserve existing query params	DON'T: Overwrite all query params on navigation when only one needed to change

Section 7 — Debugging Tips

Bug 1 — RouterLink clicks do nothing / full page reloads

Symptom	Clicking routerLink='/home' either does nothing or causes a full browser reload
Cause	RouterLink and RouterOutlet are directives that must be explicitly imported in standalone components. If not imported, Angular ignores the routerLink attribute.
Fix	Add RouterOutlet, RouterLink, RouterLinkActive to the component's imports array
Angular note	In older NgModule apps this was handled by RouterModule. In standalone (Angular 17+), every directive must be explicitly imported.

Bug 2 — Component shows stale data after navigating to a different record

Symptom	Navigate from /users/1 to /users/2 — URL changes correctly, but component still shows User 1's data
Cause	Angular reused the component instance. ngOnInit didn't fire again. The HTTP call was tied to snapshot in ngOnInit.
Fix	Replace snapshot with reactive param stream: user$ = this.route.paramMap.pipe(switchMap(...))
Angular note	Angular's Router decides whether to reuse or recreate based on whether the same component class handles both routes — no error, no log entry.

Bug 3 — ActivatedRoute.params gives the parent route's params, not the current one

Symptom	In a child component inside a nested route, params.get('id') returns null even though :id is in the URL
Cause	When routes are nested, each component's injected ActivatedRoute refers to that component's route segment, not the full URL.
Fix	Use this.route.parent!.paramMap.pipe(map(params => params.get('id')!))
Angular note	Each ActivatedRoute is scoped to its own segment. Use .parent traversal or restructure routes.

Section 8 — Real-World Applications

Scenario 1 — Master-Detail navigation with automatic data reload

In a CRM or admin panel, a user browses a list of contacts and clicks each one to see their details. The detail component must load fresh data for every selected contact — even when clicking between contacts without going back to the list.

```
export class ContactDetailComponent {
  private route = inject(ActivatedRoute);
  private router = inject(Router);
  private contactService = inject(ContactService);

  // Reactive: fires on every :id change, cancels previous HTTP call via switchMap
  contact$ = this.route.paramMap.pipe(
    switchMap(params => {
      const id = params.get('id')!;
      return this.contactService.getContact(id);
    }),
    takeUntilDestroyed(),
  );

  editContact(id: string): void {
    this.router.navigate(['/contacts', id, 'edit'], {
      queryParamsHandling: 'preserve',
    });
  }
}
```

Scenario 2 — Search page with shareable URL state (query params as source of truth)

```
export class ProductSearchComponent implements OnInit {
  private route = inject(ActivatedRoute);
  private router = inject(Router);
  private productService = inject(ProductService);

  searchTerm = '';

  results$ = this.route.queryParamMap.pipe(
    switchMap(params => {
      const q = params.get('q') ?? '';
      const page = Number(params.get('page') ?? '1');
      this.searchTerm = q;
      return this.productService.search(q, page);
    }),
    takeUntilDestroyed(),
  );

  onSearch(event: Event): void {
    const q = (event.target as HTMLInputElement).value;
    this.router.navigate([], {
      relativeTo: this.route,
      queryParams: { q, page: 1 },
      queryParamsHandling: 'merge',
    });
  }
}
```

Section 9 — Practice Exercises

Exercise 1 — Beginner

Create a standalone Angular app with three routes: /home, /about, and /contact. Add a navigation bar using RouterLink with RouterLinkActive that highlights the currently active link with a CSS class. Ensure unknown URLs redirect to a NotFoundComponent.

Exercise 2 — Intermediate

Build a ProductListComponent that displays a list of 5 hardcoded products. When a product is clicked, navigate programmatically to /products/:id. In ProductDetailComponent, read the :id from the route using the Observable paramMap approach and display a 'product not found' message if the ID doesn't match any product. Verify that navigating between product detail pages updates the displayed product without a full page reload.

Exercise 3 — Advanced

Build a searchable list page at /employees that stores the search term and current page number as query parameters in the URL (/employees?search=alice&page=2). The search input should

update the URL on every keystroke (debounced 300ms). The EmployeeListComponent must derive its data entirely from queryParamMap$ — no component-level state variables for the search term or page. Add Previous/Next pagination buttons that update only the page query param while preserving the search param using queryParamsHandling: 'merge'. Verify that the browser Back button correctly restores the previous search state.

Section 10 — Key Takeaways

Core concepts in plain English:

- The Router is a URL-to-component mapping engine — it reads the browser URL and decides which component to render in router-outlet
- Route parameters (:id) are always strings — always convert before use
- snapshot is a one-time read; paramMap$ is a live stream — use the stream for any component that handles multiple records
- Query parameters are optional key-value pairs after ? — ideal for filters, pagination, and shareable state
- Route matching is top-to-bottom, first-match-wins — wildcard ** must be last

Quick Reference Table:

Use Case	API
Navigate in template	[routerLink]="['/path', param]"
Navigate in code	router.navigate(['/path', param])
Navigate relative to current	router.navigate(['../sibling'], { relativeTo: this.route })
Read path param (once)	route.snapshot.paramMap.get('id')
Read path param (reactive)	route.paramMap.pipe(switchMap(...))
Read query param (reactive)	route.queryParamMap.pipe(map(...))
Add query params on navigate	router.navigate(['/path'], { queryParams: { key: val } })
Preserve existing query params	{ queryParamsHandling: 'merge' }
Active link CSS	routerLinkActive="css-class"

Angular-specific rules:

- Import RouterOutlet, RouterLink, RouterLinkActive explicitly in every standalone component that uses them
- Register routes with provideRouter(APP_ROUTES) in app.config.ts
- Always put { path: '**' } as the final route
- Use switchMap to connect paramMap$ to HTTP calls — it automatically cancels stale requests

- Always clean up manual paramMap$ subscriptions with takeUntilDestroyed()

ONE-LINE MEMORY AID: The URL is the GPS address; paramMap$ is your live tracking feed; snapshot is a screenshot — only one of them updates when you turn the corner.

Section 11 — Thought-Provoking Question + Answer

Question: If Angular reuses component instances when only URL params change (for performance), does that mean component state from the previous param value could leak into the next one? How do you prevent this?

Answer: Yes — this is a real and dangerous edge case. When you navigate from /users/1 to /users/2, Angular doesn't destroy UserDetailComponent and create a fresh one. It keeps the same instance and fires a new value through paramMap$. If your component has local state (a isLoading boolean, an errorMessage string, a form value), that state carries over from the previous navigation unless you explicitly reset it.

The switchMap operator inside paramMap$ handles the HTTP call correctly (cancels and re-fetches), but it does nothing about your component's local variables. Use a tap() at the start of the pipeline to reset local state before the new data arrives:

```
user$ = this.route.paramMap.pipe(
  tap(() => {
    // Explicitly reset local state on every navigation
    this.errorMessage = null;
    this.isEditing = false;
  }),
  switchMap(params => this.userService.getUser(params.get('id')!)),
  takeUntilDestroyed(),
);
```

Practical rule: Any local state variable in a routed component that must not persist between navigations should be reset in a tap() at the beginning of your paramMap$ pipeline, or eliminated entirely by deriving it from the stream.

Section 12 — Related Topics to Learn Next

ES6/JS Concepts:

- Browser History API (pushState, replaceState) — the Router is built on top of this; understanding it explains why deep links and back-navigation work the way they do

TypeScript Concepts:

- Discriminated unions — useful for modelling route state (loading | success | error) cleanly without flags

Angular-Specific Concepts:

• Topic 39 — Route Guards — the natural next step: controlling who can navigate where before the component ever loads

• Topic 40 — Lazy Loading — route-based code splitting: load feature module JS only when the user first navigates to that route

• Resolver (covered in Topic 39) — pre-fetching data before a route activates so the component never renders in a loading state

• Named Router Outlets — for advanced layouts with multiple simultaneous router outlets

• Router Events — NavigationStart, NavigationEnd observables for global loading indicators and analytics

Section 13 — Study Plan Checkpoint

Directly depends on: Topic 20 (RxJS Subscription Management), Topic 21 (takeUntilDestroyed), Topic 22 (async Pipe), Topic 25 (Lifecycle Hooks), Topic 27 (Services & DI), Topic 36 (HttpClient).

Directly unlocks: Topic 39 (Route Guards), Topic 40 (Lazy Loading), Topic 44 (State Management), Topic 47/48 (Testing).

Production readiness: You are ready to build navigable multi-page Angular apps with dynamic params and shareable query-param state — but hold off on nested routes and advanced guard patterns until Topic 39 fills the gaps.

Section 14 — Common Interview Questions

Q1 (Mid): What's the difference between ActivatedRoute.snapshot.paramMap and ActivatedRoute.paramMap$? When would you use each?

Strong answer covers: snapshot is a static one-time capture; paramMap$ is a live Observable; Angular reuses component instances when only params change; paramMap$ + switchMap is the correct pattern for master-detail navigation; snapshot acceptable only when component is always freshly created.

Weak answer misses: The component reuse behavior.

Q2 (Mid-Senior): How do you pass data between routes without putting it in the URL?

Strong answer covers: state object in router.navigate extras; shared service with BehaviorSubject; route data property for static metadata; Resolver for pre-fetched data; why NOT to use component inputs alone.

Weak answer misses: That state object is temporary (lost on refresh); a shared service is the production-safe answer.

Q3 (Junior-Mid): Explain pathMatch: 'full' — why does the empty path redirect break without it?

Strong answer covers: Angular Router matches by prefix by default — path: '' matches every URL; pathMatch: 'full' means entire remaining URL must equal the path exactly; without it, redirect fires for all routes including /users/42.

Weak answer misses: Explaining why prefix matching causes all routes to match the empty path.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Manual URL string construction in navigate()

```
// WRONG
this.router.navigate([`/users/${userId}/orders/${orderId}`]);

// CORRECT — array syntax, Router handles encoding
this.router.navigate(['/users', userId, 'orders', orderId]);
```

Why it breaks: If the route changes, every manual string must be hunted down. TypeScript can't catch typos in strings. Special characters in IDs aren't encoded correctly.

Anti-Pattern 2 — Subscribing to paramMap$ inside ngOnInit without cleanup

```
// WRONG — memory leak
ngOnInit(): void {
  this.route.paramMap.subscribe(params => {
    this.loadUser(params.get('id')!);
  });
}

// CORRECT
user$ = this.route.paramMap.pipe(
  switchMap(params => this.userService.getUser(params.get('id')!)),
  takeUntilDestroyed(),
);
```

Why it breaks: Subscriptions accumulate across destroyed components. Duplicate HTTP calls, state mutations on components no longer in the DOM.

Anti-Pattern 3 — Using router.navigate for external URLs

```
// WRONG — Angular Router treats this as an internal path
this.router.navigate(['https://external-site.com/report']);

// CORRECT — bypass Router for external URLs
window.open('https://external-site.com/report', '_blank');
```

Why it breaks: Angular Router strips the protocol, tries to match 'https:' as a route segment, fails silently, and never navigates anywhere.

Section 16 — Mental Model Summary

Core concept: Angular Router maps the browser URL to a component tree, re-rendering router-outlet on every navigation while keeping the shell component alive.

Key rules:

- 1. Route matching is top-to-bottom, first-match-wins — wildcard ** must be last
- 2. Route params are always string | null — convert before use
- 3. snapshot is one-time; paramMap$ is live — use the Observable for components handling multiple records
- 4. Angular reuses component instances on param-only changes — ngOnInit does not re-fire
- 5. Query params are the correct home for shareable, bookmarkable filter state
- 6. MOST IMPORTANT: Use route.paramMap.pipe(switchMap(...), takeUntilDestroyed()) — never snapshot inside ngOnInit — for any component navigating between records of the same type

Side-by-side comparison:

	snapshot.paramMap	**paramMap$ Observable**
When it fires	Once, at component creation	Every time the URL param changes
Safe for reused components	No	Yes
Requires cleanup	No	Yes — takeUntilDestroyed()
Works with async pipe	No	Yes
Best for	Static pages always freshly created	Master-detail, paginated lists

---

TOPIC 39 — Route Guards (CanActivate, CanDeactivate, Resolve)

Angular Core — HTTP & Routing | Difficulty: Intermediate | Relevance: Critical

---

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — Guards are conceptually simple (return true/false), but the real complexity lies in composing them correctly with async data, redirect logic, and the Resolve pattern which requires understanding the full navigation lifecycle.

Angular Relevance: Critical — Every production Angular app uses guards; authentication walls, unsaved-changes protection, and pre-fetched data are all guard responsibilities — getting them wrong silently exposes protected routes or breaks navigation flow.

Section 1 — Explanation

THE NIGHTCLUB BOUNCER ANALOGY

Imagine every route in your app is a room in a nightclub.

- CanActivate is the bouncer at the door — before you enter, they check your ID. If you're on the list, you get in. If not, you're redirected to the entrance desk.
- CanDeactivate is the coat check attendant who stops you on the way OUT — 'Wait, did you leave anything behind? You have unsaved changes — are you sure you want to leave?'
- Resolve is the venue staff who sets up the table BEFORE you're seated — by the time you walk in, the food is already on the table. The component receives fully-loaded data the moment it mounts.

TECHNICAL EXPLANATION

Route guards are functions (in modern Angular) that the Router calls during the navigation lifecycle before committing to a route change. They return: true (allow), false (block), UrlTree (block and redirect), or Observable/Promise of the above.

Navigation lifecycle order:

```
URL change triggered
  -> CanDeactivate (can we leave the current route?)
  -> CanActivate (can we enter the new route?)
  -> Resolve (fetch data before component mounts)
  -> Component renders with data already available
```

QUICK REFERENCE TABLE — Guard Types:

Guard	Runs When	Returns	Primary Use Case
canActivate	Before entering a route	boolean | UrlTree	Auth walls, role checks

canActivateChild	Before entering any child route	boolean | UrlTree	Protect entire feature sections
canDeactivate	Before leaving a route	boolean	Unsaved changes warning
canMatch	Before matching a route at all	boolean	Show different components per role
resolve	After guards pass, before component mounts	any (resolved data)	Pre-fetch required data

Section 2 — Connection to Prior Concepts

Directly builds on:

- Topic 38 — Angular Router: Guards are attached to the same Routes array
- Topic 8 — Promises & async/await: Guards can return Promises
- Topic 20 & 22 — RxJS & async Pipe: Guards can return Observables
- Topic 27 — Services & DI: Every guard uses inject() for auth services, router, and data services
- Topic 36 — HttpClient: The Resolve guard's purpose is to wrap an HTTP call and deliver its result before the component mounts

Bridging note: In Topic 38 you learned that the Router maps URLs to components. Guards are the policy layer inserted into that mapping — they answer 'should this mapping be allowed right now?' before any component lifecycle runs.

Single most-direct extension of: Topic 38 — Angular Router.

Section 3 — Code Example

CanActivate — Auth guard:

```
// guards/auth.guard.ts
export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);

  return auth.isLoggedIn$.pipe(
    map(isLoggedIn => {
      if (isLoggedIn) {
        return true;
      }
      // Return UrlTree — blocks AND redirects atomically
      return router.createUrlTree(['/login'], {
        queryParams: { returnUrl: state.url }
      });
    }),
  );
};
```

Attaching the guard to a route:

```
// app.routes.ts
{
  path: 'dashboard',
  component: DashboardComponent,
  canActivate: [authGuard],
},
{
  path: 'admin',
  component: AdminComponent,
  canActivate: [authGuard, roleGuard], // both must return true
},
```

CanDeactivate — Unsaved changes guard:

```
export interface HasUnsavedChanges {
  hasUnsavedChanges(): boolean;
}

export const unsavedChangesGuard: CanDeactivateFn<HasUnsavedChanges> =
  (component) => {
    if (component.hasUnsavedChanges()) {
      return confirm('You have unsaved changes. Leave anyway?');
    }
    return true;
  };

// In the component:
export class EditProfileComponent implements HasUnsavedChanges {
  private form = inject(FormBuilder).group({ name: [''], email: [''] });

  hasUnsavedChanges(): boolean {
    return this.form.dirty;
  }
}
```

Resolve — Pre-fetch data before component mounts:

```
export const userResolver: ResolveFn<User> = (route) => {
  const userService = inject(UserService);
  const router = inject(Router);
  const id = route.paramMap.get('id')!;

  return userService.getUser(id).pipe(
    catchError(() => {
      router.navigate(['/not-found']);
      return EMPTY;
    }),
  );
};

// Route config:
{
  path: 'users/:id',
  component: UserDetailComponent,
  resolve: { user: userResolver },
}

// Component — synchronous access:
user = this.route.snapshot.data['user'] as User;
```

Section 4 — Before & After Refactor

BEFORE — Auth check inside the component itself:

```
// WRONG: component mounts first, then checks auth
ngOnInit(): void {
  if (!this.auth.isLoggedIn()) {
    this.router.navigate(['/login']);
  }
  // Problem: sensitive UI flashes on screen for logged-out users
}
```

AFTER — Auth check in a guard (component never mounts until approved):

```
// CORRECT: guard runs BEFORE component is created
export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);
  return auth.isLoggedIn$.pipe(
    map(isLoggedIn => isLoggedIn || router.createUrlTree(['/login']))
  );
};
```

Why this refactor matters: Guards run in the Router's navigation pipeline before any component lifecycle begins. The component's constructor, ngOnInit, and template never execute unless the guard returns true.

Section 5 — Common Mistakes & Misconceptions

Mistake 1 — Using router.navigate() inside a guard instead of returning a UrlTree

The mistake	Calling this.router.navigate(['/login']) inside a guard and returning false
Why developers make it	It feels like the natural way to redirect — the same pattern used in components
How to avoid it	Always return router.createUrlTree(['/login'])
Angular consequence	Triggers a second navigation event while the first is still in progress — race conditions, duplicate history entries, console warnings

Mistake 2 — Returning EMPTY from a Resolver without redirecting

The mistake	return EMPTY in a resolver when the resource isn't found, without navigating elsewhere

Why developers make it	EMPTY stops the stream cleanly — feels like the right 'nothing to return' value
How to avoid it	Always pair return EMPTY with router.navigate(['/not-found']) before it
Angular consequence	EMPTY completes the resolver Observable without emitting. The URL changes but the component never mounts — blank page with no feedback

Mistake 3 — Treating canActivate returning false as a redirect

The mistake	Returning false from a guard and expecting the user to land somewhere useful
Why developers make it	'False means blocked, so the router will handle it' — but the Router just cancels
How to avoid it	Always return a UrlTree when you want to redirect. Return false only to block with no redirect.
Angular consequence	URL reverts to previous URL silently. Users are confused — they clicked a link and nothing happened.

Section 6 — Do's and Don'ts

DO	DON'T
DO: Return router.createUrlTree(['/login']) from guards to redirect	DON'T: Call router.navigate() inside a guard — it triggers a race condition
DO: Write guards as plain CanActivateFn functions using inject()	DON'T: Write guards as classes with implements CanActivate — deprecated
DO: Redirect to a returnUrl query param so users land back after login	DON'T: Redirect always to /home after login — users lose their intended destination
DO: Use resolve to pre-fetch data the component always needs on mount	DON'T: Use Resolve for optional data — it delays navigation even when data isn't critical
DO: Implement HasUnsavedChanges interface on components guarded by canDeactivate	DON'T: Read component state directly inside a guard — breaks separation of concerns
DO: Handle resolver errors with catchError and redirect to a 404 page	DON'T: Let resolver errors propagate — silent navigation cancellation leaves users stranded

Section 7 — Debugging Tips

Bug 1 — Guard runs but navigation completes anyway (guard seems ignored)

Symptom	Guard returns false or a UrlTree but the protected component still mounts
Cause	The guard function is defined but not attached to the route in app.routes.ts. The route has canActivate: [] (empty array).
Fix	Check route config: { path: 'dashboard', component: DashboardComponent, canActivate: [authGuard] }
Angular note	Angular gives no warning when canActivate is an empty array — it simply runs no guards.

Bug 2 — Resolved data is undefined in the component

Symptom	this.route.snapshot.data['user'] is undefined even though the resolver ran
Cause	The key used to access route.data doesn't match the key used in the route config's resolve object. Keys are case-sensitive.
Fix	resolve: { user: userResolver } -> access as route.snapshot.data['user'] — exact same key
Angular note	TypeScript won't catch this mismatch because route.snapshot.data is typed as { [key: string]: any }.

Bug 3 — CanDeactivate guard fires even on programmatic navigation

Symptom	confirm() dialog appears when the app programmatically navigates away after a successful form save
Cause	CanDeactivate fires on ALL navigation away from the route — including router.navigate() calls from your own code.
Fix	Use a flag on the component: isSaving = false; set to true before programmatic navigate-away; check it in hasUnsavedChanges()
Angular note	CanDeactivate is a navigation lifecycle hook, not a user-interaction hook. The component must communicate its intent.

Section 8 — Real-World Applications

Scenario 1 — Role-based access with redirect to correct landing page

```
export const roleGuard: CanActivateFn = (route: ActivatedRouteSnapshot) => {
  const auth = inject(AuthService);
  const router = inject(Router);
  const requiredRole = route.data['requiredRole'] as string;

  return auth.currentUser$.pipe(
    map(user => {
      if (!user) return router.createUrlTree(['/login']);
      if (hasRole(user.role, requiredRole)) return true;
      return router.createUrlTree([getFallbackRoute(user.role)]);
    }),
  );
};

// Route config:
{
  path: 'admin',
  component: AdminComponent,
  canActivate: [authGuard, roleGuard],
  data: { requiredRole: 'admin' },
},
```

Scenario 2 — Resolver with error handling for a product detail page

```
export const productResolver: ResolveFn<Product> = (route) => {
  const productService = inject(ProductService);
  const router = inject(Router);
  const notify = inject(NotificationService);
  const slug = route.paramMap.get('slug')!;

  return productService.getBySlug(slug).pipe(
    catchError(err => {
      notify.error(err.status === 404 ? 'Product not found' : 'Unable to load
product');
      router.navigate(['/catalog']);
      return EMPTY;
    }),
  );
};

// Component — zero loading state needed:
product = this.route.snapshot.data['product'] as Product;
```

Section 9 — Practice Exercises

Exercise 1 — Beginner: Create an authGuard using CanActivateFn that checks a simple AuthService with a boolean isLoggedIn property. If the user is not logged in, redirect them to /login using router.createUrlTree(). Apply the guard to a /dashboard route and verify in the browser that navigating to /dashboard while logged out redirects correctly.

Exercise 2 — Intermediate: Build a CanDeactivate guard for a SettingsFormComponent that uses a Reactive Form. The guard must ask the component whether it has unsaved changes via a hasUnsavedChanges() method. Replace the native confirm() dialog with a custom Angular modal component that returns an Observable boolean — the guard must return this Observable directly to the Router.

Exercise 3 — Advanced: Build a feature section at /projects/:id that uses three guards in combination: (1) authGuard to require login, (2) projectAccessGuard that reads the :id param and calls a ProjectService.canAccess(id) API endpoint, (3) a projectResolver that fetches the full project data. Handle the case where the project API returns 404 by redirecting to a /not-found route with a query param ?resource=project. Ensure all Observables are correctly handled and no memory leaks exist.

Section 10 — Key Takeaways

Core concepts in plain English:

- Guards run before the component lifecycle — components never mount if a guard blocks
- CanActivate controls entry; CanDeactivate controls exit; Resolve pre-loads data
- Modern guards are plain functions (CanActivateFn) using inject() — not classes
- Always return UrlTree to redirect — never call router.navigate() inside a guard
- Resolve eliminates loading spinners by making data available synchronously on component mount
- Returning EMPTY from a resolver without redirecting leaves users on a blank page

Angular-specific rules:

- Write guards as CanActivateFn / CanDeactivateFn functions — class-based guards are deprecated
- Use route.data to pass configuration (like requiredRole) into guards
- Resolver errors must be caught — unhandled errors silently cancel navigation
- Multiple guards in an array run sequentially — if any returns false/UrlTree, subsequent guards do not run
- canActivateChild on a parent route protects all its children without repeating the guard

ONE-LINE MEMORY AID: Guards are the Router's bouncers — CanActivate checks ID at the door, CanDeactivate checks for forgotten items on the way out, and Resolve sets the table before you're seated.

Section 11 — Thought-Provoking Question + Answer

Question: A Resolve guard pre-fetches data before a component mounts — but this means the user sees no visual feedback while waiting for that API call. Is that actually better UX than a loading spinner inside the component, and when should you choose one over the other?

Answer: Both approaches have the same wall-clock time. The difference is where the wait is experienced. With a Resolver, the user stays on the previous page while data loads. With an in-component loading spinner, the user navigates immediately to a partially rendered shell.

Resolvers shine for short, fast API calls (<300ms) where a spinner would be a distracting flash. They are poor choices for slow operations (>1s) because the user gets no feedback — the previous page appears frozen.

Practical rule: Use Resolve for fast, required data where a loading flash would hurt UX. Use in-component loading for slow calls, optional data, or when rich error recovery UI is needed. Combine both: Resolve for critical data, in-component for supplementary data loaded after mount.

Section 12 — Related Topics to Learn Next

ES6/JS: Promise.all / forkJoin — multiple resolvers run in parallel.

TypeScript: Discriminated unions — model guard return states for testable guard logic.

Angular-Specific:

- Topic 40 — Lazy Loading: Guards on lazy-loaded routes run before the module/component bundle is downloaded
- Topic 44 — State Management: Guards often read auth state from a signal or BehaviorSubject store
- Topic 47 — Testing: Testing guards with RouterTestingHarness and mocked services is a core interview skill
- canMatch guard — determines whether a route is even considered during matching (useful for A/B testing)

Section 13 — Study Plan Checkpoint

Directly depends on: Topic 8 (Promises), Topic 20 (RxJS), Topic 27 (DI), Topic 36 (HttpClient), Topic 38 (Angular Router).

Directly unlocks: Topic 40 (Lazy Loading), Topic 44 (State Management), Topic 47/48 (Testing).

Production readiness: You are ready to implement auth walls, role checks, and unsaved-changes protection in real apps — but test your guard logic with mocked services before deploying.

Section 14 — Common Interview Questions

Q1 (Mid): What's the difference between returning false and returning a UrlTree from a CanActivate guard?

Strong answer covers: false blocks navigation and leaves the user on the current URL with no indication; UrlTree blocks AND triggers a redirect; router.createUrlTree() is the correct way to create a UrlTree; calling router.navigate() inside a guard causes a race condition.

Weak answer misses: The race condition caused by calling navigate() inside a guard.

Q2 (Mid-Senior): Why would you use a Resolve guard instead of fetching data inside ngOnInit? What are the trade-offs?

Strong answer covers: Resolver runs before component mounts — data available synchronously in snapshot.data; eliminates loading states for fast API calls; trade-off: user sees no navigation progress for slow resolvers; removes component's ability to handle its own loading/error UI.

Weak answer misses: The UX trade-off of slow resolvers appearing as frozen navigation.

Q3 (Senior): How would you protect an entire feature section (multiple child routes) with a single auth guard?

Strong answer covers: Use canActivateChild on the parent route; canActivateChild runs on every child navigation; difference between canActivate (runs once entering parent) vs canActivateChild (runs on every child navigation).

```
{
  path: 'admin',
  canActivateChild: [authGuard],
  children: [
    { path: 'users', component: AdminUsersComponent },
    { path: 'settings', component: AdminSettingsComponent },
  ]
}
```

Weak answer misses: The distinction between canActivate and canActivateChild.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Class-based guards (deprecated pattern)

```
// WRONG — deprecated since Angular 15
@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {
  constructor(private auth: AuthService, private router: Router) {}
  canActivate(): Observable<boolean | UrlTree> { ... }
}

// CORRECT — functional guard
export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);
  return auth.isLoggedIn$.pipe(
    map(loggedIn => loggedIn || router.createUrlTree(['/login']))
  );
};
```

Anti-Pattern 2 — Resolver that never handles errors

```
// WRONG — unprotected resolver
export const projectResolver: ResolveFn<Project> = (route) => {
  return inject(ProjectService).getProject(route.paramMap.get('id')!);
  // No catchError — API failure silently strands the user
};

// CORRECT
return projectService.getProject(id).pipe(
  catchError(() => {
    router.navigate(['/error'], { queryParams: { code: 'not-found' } });
    return EMPTY;
  }),
);
```

Anti-Pattern 3 — Checking auth state synchronously from localStorage

```
// WRONG — token presence != token validity
export const authGuard: CanActivateFn = () => {
  const token = localStorage.getItem('auth_token');
  if (!token) { inject(Router).navigate(['/login']); return false; }
  return true; // expired tokens bypass this guard
};

// CORRECT — validate via AuthService
export const authGuard: CanActivateFn = () => {
  return inject(AuthService).isAuthenticated$.pipe(
    map(valid => valid || inject(Router).createUrlTree(['/login']))
  );
};
```

Section 16 — Mental Model Summary

Core concept: Route guards are policy functions that run inside the Router's navigation pipeline — before any component mounts — to control whether navigation proceeds, redirects, or pre-loads data.

Key rules:

- 1. Guards run before component lifecycle — components never mount if a guard blocks
- 2. Return UrlTree to redirect; false only to block with no redirect
- 3. Never call router.navigate() inside a guard — return UrlTree instead
- 4. Resolve runs after CanActivate passes — data is synchronously available on mount
- 5. Always catchError in Resolvers — unhandled errors silently cancel navigation
- 6. MOST IMPORTANT: Return router.createUrlTree(['/login']) from a guard to redirect — never call router.navigate() inside a guard as it triggers a race condition

	CanActivate	CanDeactivate	Resolve
us	Before entering	Before leaving	After activate, before mount
cks	Entry	Exit	N/A (delays mount)
urns	boolean | UrlTree	boolean	Observable<T>
mmon use	Auth / role checks	Unsaved changes	Pre-fetch required data
mponent state	No component yet	Component is alive	No component yet

TOPIC 40 — Lazy Loading & Route-Based Code Splitting

Angular Core — HTTP & Routing | Difficulty: Intermediate | Relevance: Critical

---

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — The configuration is concise, but understanding why it works requires grasping how JavaScript module bundling, the browser's network loading model, and Angular's dependency injection system all interact at the route boundary.

Angular Relevance: Critical — Every production Angular app with more than a handful of routes should use lazy loading; it is the single highest-impact performance optimisation available at the architectural level, and Angular's Router makes it nearly free to implement.

Section 1 — Explanation

THE LIBRARY BRANCH ANALOGY

Imagine a large public library system with a central branch and several specialist branches (law, medicine, science, children's).

Without lazy loading: Every time anyone walks into the central library, staff wheel out every book from every branch and stack them in the lobby — just in case someone might ask for them. The lobby is packed and the first patron waits 3 minutes before they can even ask a question.

With lazy loading: The central library keeps only its own books in the lobby. The specialist branches stay at their own locations. When a patron specifically asks for a law book, a courier is dispatched to the law branch and retrieves just what's needed.

- The central library = your initial JavaScript bundle — everything loaded on first visit
- Each specialist branch = a feature area (admin panel, settings, reports)
- The courier dispatch = loadComponent / loadChildren in your route config
- The books arriving = JavaScript chunk files downloaded on demand from your server

TECHNICAL EXPLANATION

When you build an Angular app, the compiler bundles your TypeScript into JavaScript files. Without lazy loading, everything goes into one main.js bundle. Lazy loading tells the bundler: 'This component/module is only needed when the user navigates to this route — put it in a separate chunk file.'

Two modern approaches in Angular 17+:

Approach	What It Does
loadComponent	Lazily loads a single standalone component
loadChildren	Lazily loads an entire routes file (feature routes)

What happens at runtime:

- 1. User visits / — browser downloads main.js only
- 2. User navigates to /admin — Router sees loadComponent, triggers dynamic import()
- 3. Browser downloads admin-dashboard-HASH.js chunk
- 4. Router instantiates AdminDashboardComponent inside router-outlet
- 5. All subsequent /admin/* navigations use the already-cached chunk

QUICK REFERENCE TABLE:

Approach	Syntax	Best For	Chunk Contains
loadComponent	loadComponent: () => import('./x').then(m => m.XComponent)	Single standalone component	That component + unique deps
loadChildren	loadChildren: () => import('./x.routes').then(m => m.X_ROUTES)	Entire feature section	All components in that routes file
Eager (default)	component: MyComponent	Shell, login, home	Added to main.js
Preloading	withPreloading(PreloadAllModules)	Routes likely to be visited	Silently prefetched after initial load

Section 2 — Connection to Prior Concepts

- Topic 38 — Angular Router: Lazy loading is a property of the route configuration
- Topic 39 — Route Guards: Guards run before the lazy chunk is downloaded — a blocked guard means the chunk is never fetched
- Topic 27 — Services & DI: Lazy-loaded routes create a new DI scope
- Topic 12 — Modules & Imports/Exports: Dynamic import() is a JavaScript module system feature
- Topic 24 — OnPush Change Detection: Lazy loading + OnPush compound — lazy loading reduces download time; OnPush reduces runtime rendering cost

Bridging note: In Topics 38 and 39 you learned that the Router maps URLs to components and guards gate that mapping. Lazy loading adds one more capability: the Router can defer even knowing about a component until the user actually needs it. Everything else — guards, resolvers, outlets — works identically.

Single most-direct extension of: Topic 38 — Angular Router.

Section 3 — Code Example

Plain JS — Dynamic import in isolation (the underlying mechanism):

```
// Static import — loaded immediately, part of the main bundle
import { AdminDashboard } from './admin-dashboard';

// Dynamic import — returns a Promise, loaded on demand
async function loadAdminWhenNeeded() {
  const module = await import('./admin-dashboard');
  return module.AdminDashboard;
}
// The bundler sees import() and creates a separate chunk file
```

loadComponent — single standalone component:

```
export const APP_ROUTES: Routes = [
  // Eager — HomeComponent included in main.js
  { path: 'home', component: HomeComponent },

  // Lazy — AdminDashboardComponent in its own chunk
  {
    path: 'admin',
    loadComponent: () =>
      import('./admin/admin-dashboard.component')
        .then(m => m.AdminDashboardComponent),
    canActivate: [authGuard], // guard runs BEFORE chunk is downloaded
  },

  // Lazy with default export
  {
    path: 'settings',
    loadComponent: () => import('./settings/settings.component'),
  },
];
```

loadChildren — entire feature section:

```
// app.routes.ts
{
  path: 'users',
  loadChildren: () =>
    import('./users/users.routes').then(m => m.USERS_ROUTES),
  canActivate: [authGuard],
},
// users/users.routes.ts
export const USERS_ROUTES: Routes = [
  { path: '', loadComponent: () => import('./user-list.component')... },
  { path: ':id', loadComponent: () => import('./user-detail.component')... },
  { path: ':id/edit', loadComponent: () => import('./user-edit.component')...,
    canDeactivate: [unsavedChangesGuard] },
];
```

app.config.ts with preloading:

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(
      APP_ROUTES,
      withPreloading(PreloadAllModules), // silently prefetch after initial load
    ),
  ],
};
```

Route-level service scoping:

```
export const USERS_ROUTES: Routes = [{
  path: '',
  providers: [UserStateService], // scoped DI context — not shared with root
  children: [
    { path: '', loadComponent: () => import('./user-list.component')... },
    { path: ':id', loadComponent: () => import('./user-detail.component')... },
  ],
}];
```

Section 4 — Before & After Refactor

BEFORE — Everything eager, one giant bundle:

```
// All components imported statically at the top — ALL go into main.js
import { HomeComponent } from './home/home.component';
import { AdminDashboardComponent } from './admin/admin-dashboard.component';
import { AdminUsersComponent } from './admin/admin-users.component';
// ...20 more imports

export const APP_ROUTES: Routes = [
  { path: 'home', component: HomeComponent },
  { path: 'admin', component: AdminDashboardComponent },
  // ...
];
```

AFTER — Feature sections lazy-loaded:

```
// No static imports of feature components
import { HomeComponent } from './home/home.component'; // small, needed immediately

export const APP_ROUTES: Routes = [
  { path: 'home', component: HomeComponent },
  {
    path: 'admin',
    canActivate: [authGuard],
    loadChildren: () => import('./admin/admin.routes').then(m => m.ADMIN_ROUTES),
  },
  {
    path: 'users',
    loadChildren: () => import('./users/users.routes').then(m => m.USERS_ROUTES),
  },
];
```

Why this refactor matters: Replacing import { X } from './x' at the top of the file with () => import('./x') inside a route config is the signal that tells the bundler to split at that boundary. The change is minimal — but the impact on initial load time in large apps is significant.

Section 5 — Common Mistakes & Misconceptions

Mistake 1 — Statically importing the component AND using loadComponent

The mistake	import { AdminComponent } from './admin.component' at top of file AND loadComponent: () => import('./admin.component')... in route
Why developers make it	IDE auto-import adds the static import; the developer doesn't notice

How to avoid it	If you use loadComponent/loadChildren, there must be NO static import of those components at the top of app.routes.ts
Angular consequence	The static import causes the bundler to include the component in main.js anyway — zero performance benefit, no error, no warning

Mistake 2 — Providing a service in both root and a lazy route's providers array

The mistake	@Injectable({ providedIn: 'root' }) on a service that is also listed in a lazy route's providers: [MyService]
Why developers make it	Developers add providers to the route for scoping without removing providedIn: 'root' from the service decorator
How to avoid it	Choose one: either providedIn: 'root' or providers: [MyService] on the route. Never both.
Angular consequence	Two instances of the service exist simultaneously. State updates in one are invisible to the other.

Mistake 3 — Assuming lazy-loaded components are re-downloaded on every navigation

The mistake	Adding artificial caching logic or worrying about performance on repeated visits to a lazy route
Why developers make it	'Lazy' sounds like 'slow every time' — developers assume the chunk is re-fetched each visit
How to avoid it	Dynamic import() is cached by the browser after the first download. Second navigation to a lazy route is as fast as an eager route.
Angular consequence	Unnecessary complexity — cache-busting logic, manual preloading of already-cached chunks. Wasted dev time.

Section 6 — Do's and Don'ts

DO	DON'T
DO: Use loadChildren for entire feature sections with multiple routes	DON'T: Import feature components statically in app.routes.ts — it defeats the lazy split
DO: Use loadComponent for single large components that are rarely visited	DON'T: Use loadComponent for every route indiscriminately — tiny components add HTTP overhead

DO: Keep eagerly loaded components minimal: shell, nav, login, home	DON'T: Lazy-load login or the root AppComponent — they're needed immediately
DO: Add canActivate: [authGuard] before loadChildren — guard blocks chunk download for unauthorized users	DON'T: Download the admin bundle before checking if the user is an admin
DO: Use withPreloading(PreloadAllModules) to background-fetch lazy chunks after initial load	DON'T: Assume all users have fast connections
DO: Scope feature-specific services in the route's providers array	DON'T: Use providedIn: 'root' for services that should only be used inside one lazy feature

Section 7 — Debugging Tips

Bug 1 — Lazy route navigates but shows a blank router-outlet with no error

Symptom	Navigation to /admin changes the URL, no console errors, but the outlet is empty
Cause	The loadChildren path resolves correctly but the exported Routes constant name in .then(m => m.X_ROUTES) doesn't match the actual export name
Fix	Check the exact export name: export const USERS_ROUTES in routes file must match .then(m => m.USERS_ROUTES) in app.routes.ts
Angular note	TypeScript won't catch this mismatch by default since module exports are typed as any.

Bug 2 — Lazy chunk is included in main.js instead of a separate file

Symptom	Angular build output shows no separate chunk files. main.js size is the same as before adding lazy routes.
Cause	A static import of the component exists somewhere in a file that is part of the eager bundle — most commonly an auto-added import at the top of app.routes.ts
Fix	Remove the static import statement. Run ng build and inspect chunk output to verify.
Angular note	Run ng build --stats-json and open with webpack-bundle-analyzer to visually confirm splits are working.

Bug 3 — NullInjectorError for a service inside a lazy-loaded component

Symptom	NullInjectorError: No provider for UserStateService appears when navigating to a lazy route
Cause	The service is provided in the lazy route's providers array but the component is accessing it via a parent injector that doesn't have it
Fix	Ensure the component consuming the service is inside the route subtree that provides it via the providers array
Angular note	Route-level providers create an environment injector scoped to that route subtree. Components outside that subtree cannot access it.

Section 8 — Real-World Applications

Scenario 1 — Admin portal with auth-gated lazy loading

```
export const APP_ROUTES: Routes = [
  { path: '', component: LandingPageComponent },
  { path: 'login', component: LoginComponent },
  {
    path: 'app',
    canActivate: [authGuard], // 1. Check auth FIRST
    loadChildren: () => // 2. THEN download portal chunk
      import('./portal/portal.routes').then(m => m.PORTAL_ROUTES),
  },
];

// portal.routes.ts
export const PORTAL_ROUTES: Routes = [{
  path: '',
  component: PortalShellComponent,
  children: [
    { path: 'dashboard', loadComponent: () => import('./dashboard')... },
    { path: 'users', loadChildren: () => import('./users/users.routes')... },
    { path: 'billing', loadChildren: () => import('./billing/billing.routes')... },
  ],
}];
```

Scenario 2 — Preloading strategy for a content site

```
@Injectable({ providedIn: 'root' })
export class SelectivePreloadingStrategy implements PreloadingStrategy {
  preload(route: Route, load: () => Observable<unknown>): Observable<unknown> {
    if (route.data?.['preload'] === true) {
      return timer(2000).pipe(switchMap(() => load()));
    }
    return of(null);
  }
}

// Route config:
{
  path: 'articles',
  loadChildren: () => import('./articles/articles.routes')...,
  data: { preload: true }, // flagged for preloading
},
{
  path: 'admin',
  loadChildren: () => import('./admin/admin.routes')...,
  data: { preload: false }, // only visited by small % — skip
  canActivate: [authGuard],
},
```

Section 9 — Practice Exercises

Exercise 1 — Beginner: Take an existing Angular app with three eagerly-loaded routes (/home, /about, /contact). Convert /about and /contact to use loadComponent. Run ng build before and after and compare the build output. Confirm in the browser's Network tab that the about chunk is only downloaded when navigating to /about.

Exercise 2 — Intermediate: Build a feature section at /products using loadChildren pointing to a products.routes.ts file. Add three lazy-loaded child routes: /products (list), /products/:id (detail), /products/new (create form). Add a ProductStateService provided at the products route level (not root). Verify that navigating away from /products and back creates a fresh service instance.

Exercise 3 — Advanced: Build a role-based lazy loading architecture with two user roles: user and admin. Route /dashboard should lazy-load UserDashboardComponent for regular users and AdminDashboardComponent for admins — using a single /dashboard URL for both. Use the canMatch guard to select which component is loaded based on the authenticated user's role. Implement a custom preloading strategy that preloads the correct dashboard chunk for the current user's role 3 seconds after login. Verify bundle splitting in the build output.

Section 10 — Key Takeaways

• Lazy loading splits your app into separate JS chunk files downloaded only when needed

• loadComponent for one standalone component; loadChildren for an entire feature routes file

• The mechanism is standard JavaScript dynamic import() — Angular's Router exploits module boundaries

• Guards run before chunks are downloaded — unauthorized users never download protected code

• Preloading silently fetches lazy chunks after initial load — fast first load WITH instant subsequent navigation

• A static import at the top of your routes file defeats the lazy split entirely

	Eager
In main.js	Yes
Separate chunk	No
Guard interaction	N/A
Best for	Shell, login, home
Child routes	Yes

ONE-LINE MEMORY AID: Lazy loading is the art of not packing books you haven't asked for yet — loadComponent for one book, loadChildren for a whole bookcase, and withPreloading for the librarian who fetches the next likely book while you're still reading the first.

Section 11 — Thought-Provoking Question + Answer

Question: If lazy loading improves initial load time by deferring chunks, but preloading fetches those chunks shortly after anyway — are they not just achieving the same result with extra steps? What is the actual user experience difference?

Answer: They achieve the same final state (all chunks downloaded) but at completely different times relative to user interaction. The distinction is the critical path.

Without lazy loading: download main.js (large) -> parse -> render -> interactive.

With lazy loading + preloading: download main.js (small) -> parse -> render -> interactive -> (background) download other chunks.

The user reaches 'interactive' in the first scenario only after the full large bundle parses. On a 3G mobile connection with 800ms latency, the difference between a 2MB main.js and a 200KB main.js is 5-6 seconds of blank screen versus a functional app.

Practical rule: Always lazy-load large feature sections. Always add withPreloading(PreloadAllModules) or a selective strategy. The combination gives you fast initial load AND instant subsequent navigation — they are complementary, not redundant.

Section 12 — Related Topics to Learn Next

ES6/JS: Dynamic import() in depth — understanding how the browser's module registry caches dynamic imports explains why lazy chunks are never re-downloaded.

TypeScript: TypeScript path aliases (@app/, @features/) — large apps with lazy loading benefit from clean import paths.

Angular-Specific:

- Topic 45 — Performance Optimisation: Lazy loading is the first layer; deferrable views, trackBy, and pure pipes are the next layers
- Angular's @defer block — a template-level lazy loading mechanism that defers component rendering until in the viewport

Section 13 — Study Plan Checkpoint

Directly depends on: Topic 12 (Modules & Imports), Topic 27 (DI), Topic 38 (Angular Router), Topic 39 (Route Guards).

Directly unlocks: Topic 45 (Performance Optimisation), Topic 44 (State Management), Topic 47/48 (Testing).

Production readiness: You are ready to implement lazy loading in real apps — but run ng build after every change and inspect the chunk output to confirm splits are actually happening.

Section 14 — Common Interview Questions

Q1 (Junior-Mid): What is lazy loading in Angular and how does it affect the build output?

Strong answer covers: Lazy loading defers downloading JS for a route; implemented via loadComponent or loadChildren with dynamic import(); bundler creates separate chunk files; main.js only contains eagerly-loaded code; first visit downloads chunk, subsequent visits use browser cache; practical impact: smaller initial bundle -> faster TTI.

Weak answer misses: Explaining how the bundler knows to split — the static vs dynamic import() distinction.

Q2 (Mid-Senior): How do route guards interact with lazy-loaded routes? Does the guard prevent the chunk from being downloaded?

Strong answer covers: Guards in canActivate run before loadChildren/loadComponent is called; a guard returning false/UrlTree means dynamic import() is never triggered; the chunk is not downloaded; unauthorized users never download admin/protected code.

Weak answer misses: Explicitly confirming the chunk is not downloaded on guard failure.

Q3 (Senior): What is a preloading strategy and when would you use a custom one over PreloadAllModules?

Strong answer covers: Preloading silently downloads lazy chunks after initial app load during browser idle time; PreloadAllModules downloads all lazy chunks; custom strategies allow

selective preloading based on route data flags, user role, network conditions; withPreloading() in provideRouter() registers the strategy.

Weak answer misses: The custom strategy use case — when PreloadAllModules causes unnecessary bandwidth on mobile.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Lazy-loading every route including small, always-visited components

```
// WRONG — lazy-loading tiny always-needed components
{ path: 'home', loadComponent: () => import('./home/home.component') }, // 3KB
{ path: 'login', loadComponent: () => import('./login/login.component') }, // always needed

// CORRECT — eager for small/always-needed, lazy for large/conditional
import { HomeComponent } from './home/home.component';
import { LoginComponent } from './login/login.component';
{ path: 'home', component: HomeComponent },
{ path: 'login', component: LoginComponent },
{ path: 'admin', loadChildren: () => import('./admin/admin.routes')... },
```

Anti-Pattern 2 — Forgetting that a static import defeats the lazy split

```
// WRONG — static import present alongside loadComponent
import { AdminDashboardComponent } from './admin/admin-dashboard.component'; // IDE added this

export const APP_ROUTES: Routes = [{
  path: 'admin',
  loadComponent: () => import('./admin/admin-dashboard.component')...
  // AdminDashboardComponent is ALREADY in main.js due to the static import above
}];

// CORRECT — zero static imports of lazy components
export const APP_ROUTES: Routes = [{
  path: 'admin',
  loadComponent: () => import('./admin/admin-dashboard.component').then(m => m.AdminDashboardComponent),
}];
```

Anti-Pattern 3 — Using loadChildren pointing to a component instead of a routes file

```
// WRONG — loadChildren expects a Routes array, not a component
{
  path: 'users',
  loadChildren: () => import('./users/user-list.component').then(m =>
m.UserListComponent),
}
// CORRECT — use the right tool
// loadComponent -> resolves to a Component class
{ path: 'users', loadComponent: () => import('./users/user-list.component')... }
// loadChildren -> resolves to a Routes array
{ path: 'users', loadChildren: () => import('./users/users.routes').then(m =>
m.USERS_ROUTES) }
```

Section 16 — Mental Model Summary

Core concept: Lazy loading replaces static imports with dynamic import() calls in route config, telling the bundler to split feature code into separate chunks that the browser downloads only when the user first navigates there.

- 1. loadComponent for one standalone component; loadChildren for an entire feature routes file
- 2. No static import of the component at the top of the routes file — it defeats the split
- 3. Guards run before the chunk is downloaded — blocked guards prevent the network request
- 4. Chunks are cached after the first download — second visit to a lazy route is as fast as eager
- 5. Route-level providers create a DI scope scoped to that route subtree
- 6. MOST IMPORTANT: A static import of a lazy component in your routes file silently defeats the entire lazy split — always remove auto-added imports and verify chunks appear in the build output

---

TOPIC 41 — Custom Directives

Angular Advanced | Difficulty: Intermediate | Relevance: High

---

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — The decorator and class structure are straightforward, but understanding when to build a directive versus a component, and how to correctly interact with the host element without breaking Angular's rendering model, requires deliberate practice.

Angular Relevance: High — Custom directives are the correct tool for reusable DOM behaviour that doesn't need its own template — click-outside detection, auto-focus, permission-based visibility, tooltip anchoring, and input formatting all belong in directives, not components.

Section 1 — Explanation

THE STICKER SHEET ANALOGY

Imagine you're decorating notebooks. A component is like printing a completely new page and inserting it — it brings its own content, its own layout, its own structure. A directive is like a sticker you apply to an existing page — it adds behaviour, changes appearance, or reacts to interaction, but the page itself already exists.

- A tooltip sticker — hover over the page, a bubble appears
- A highlight sticker — the text underneath turns yellow
- A lock sticker — you can't write on this section unless you have a key

In Angular terms:

- The host element is the notebook page — it already exists in the template
- Your directive class is the sticker — it attaches to that element via a CSS selector
- @HostListener is the sticker reacting to what happens on the page (hover, click, keypress)
- @HostBinding is the sticker changing something about the page's appearance or attributes

TECHNICAL EXPLANATION

A directive is an Angular class decorated with @Directive that attaches to a DOM element matched by a CSS selector. Unlike components, directives have no template of their own — they operate on the host element they are placed on.

Type	Purpose	Examples
Attribute directive	Changes appearance or behaviour of an existing element	ngClass, ngStyle, your custom appHighlight
Structural directive	Changes the DOM structure by adding/removing elements	@if, @for, *ngIf, *ngFor
Component	A directive with a template	Every @Component is technically a directive

Core building blocks:

```
@Directive({ selector: '[appMyDirective]' }) // attaches to elements with this
attribute
class MyDirective {
  @HostBinding('class.active') // binds to a property of the host element
  @HostListener('click') // listens to an event on the host element
  @Input() // receives configuration from the template
  ElementRef // direct reference to the host DOM element
  Renderer2 // safe, platform-agnostic DOM manipulation
}
```

Section 2 — Connection to Prior Concepts

- Topic 17 — TypeScript Decorators: @Directive, @HostBinding, @HostListener, and @Input are all decorators
- Topic 27 — Services & DI: ElementRef, Renderer2, and any custom service can be injected using inject()
- Topic 26 — @Input() & @Output(): Directives accept @Input() for configuration exactly as components do
- Topic 25 — Component Lifecycle Hooks: Directives have the same lifecycle hooks (ngOnInit, ngOnChanges, ngOnDestroy)
- Topic 30 — Built-in Directives: ngClass, ngStyle, and ngTemplateOutlet are Angular's own attribute directives — you've been consuming this pattern; now you'll build your own

Bridging note: Every built-in directive you used in Topic 30 (ngClass, ngStyle) is implemented using exactly the same @Directive decorator, @HostBinding, and @HostListener pattern you'll learn here. You've been consuming this API since Topic 30 — now you're on the producing side.

Single most-direct extension of: Topic 30 — Built-in Directives.

Section 3 — Code Example

Plain TS — The core pattern in isolation:

```
class HighlightBehaviour {
  private element: HTMLElement;
  private color: string = 'yellow';

  constructor(el: HTMLElement, color: string) {
    this.element = el;
    this.color = color;
  }

  onMouseEnter() { this.element.style.backgroundColor = this.color; }
  onMouseLeave() { this.element.style.backgroundColor = ''; }
}
// Angular's @Directive does this wiring automatically
// @HostListener replaces manual addEventListener
// @HostBinding replaces manual style assignment
```

Highlight directive — attribute directive fundamentals:

```
@Directive({
  selector: '[appHighlight]', // applied to any element with this attribute
  standalone: true,
})
export class HighlightDirective implements OnInit {
  @Input() appHighlight: string = 'yellow';
  @Input() defaultColor: string = 'transparent';

  @HostBinding('style.backgroundColor') backgroundColor = '';

  ngOnInit(): void {
    this.backgroundColor = this.defaultColor;
  }

  @HostListener('mouseenter')
  onMouseEnter(): void { this.backgroundColor = this.appHighlight; }

  @HostListener('mouseleave')
  onMouseLeave(): void { this.backgroundColor = this.defaultColor; }
}
```

Using the directive in a template:

```
@Component({
  standalone: true,
  imports: [HighlightDirective], // MUST be imported
  template: `
    <div appHighlight="lightblue">Hover over me</div>
    <div [appHighlight]="selectedColor" defaultColor="lightyellow">Dynamic</div>
    <p appHighlight>Default yellow highlight</p>
  `,
})
export class ProductCardComponent {
  selectedColor = 'lightgreen';
}
```

Permission directive using Renderer2 and a service:

```
@Directive({ selector: '[appHasPermission]', standalone: true })
export class HasPermissionDirective implements OnInit {
  @Input() appHasPermission!: string;

  private el = inject(ElementRef);
  private renderer = inject(Renderer2); // platform-safe DOM manipulation
  private auth = inject(AuthService);

  ngOnInit(): void {
    if (!this.auth.hasPermission(this.appHasPermission)) {
      this.renderer.setStyle(this.el.nativeElement, 'display', 'none');
    }
  }
}

// Template usage:
// <button appHasPermission="admin:write" (click)="deleteUser()">Delete</button>
```

Click-outside directive:

```
@Directive({ selector: '[appClickOutside]', standalone: true })
export class ClickOutsideDirective {
  @Output() clickOutside = new EventEmitter<void>();
  private el = inject(ElementRef);

  // 'document:click' syntax — Angular supports global event listeners
  @HostListener('document:click', ['$event.target'])
  onDocumentClick(target: HTMLElement): void {
    const isInside = this.el.nativeElement.contains(target);
    if (!isInside) { this.clickOutside.emit(); }
  }
}

// Template usage:
// <div appClickOutside (clickOutside)="closeDropdown()">
```

Section 4 — Before & After Refactor

BEFORE — DOM behaviour duplicated in every component:

```
// product-card.component.ts
@HostListener('mouseenter')
onEnter() { this.el.nativeElement.style.backgroundColor = 'lightyellow'; }

// user-card.component.ts — SAME code duplicated
@HostListener('mouseenter')
onEnter() { this.el.nativeElement.style.backgroundColor = 'lightyellow'; }
// ...and again in every other component
```

AFTER — Behaviour extracted into a reusable directive:

```
@Directive({ selector: '[appHighlight]', standalone: true })
export class HighlightDirective {
  @Input() appHighlight = 'lightyellow';
  @HostBinding('style.backgroundColor') bg = '';
  @HostListener('mouseenter') onEnter() { this.bg = this.appHighlight; }
  @HostListener('mouseleave') onLeave() { this.bg = ''; }
}

// Apply to any element in any component — zero duplication:
// <app-product-card appHighlight="lightyellow" />
// <app-user-card appHighlight="lightblue" />
// <button appHighlight>Hover me</button>
```

Why this refactor matters: Directives are Angular's mechanism for behaviour composition without inheritance. Any element in any component gains the behaviour by adding the attribute. Change the directive once — every element using it updates.

Section 5 — Common Mistakes & Misconceptions

Mistake 1 — Manipulating DOM directly via ElementRef.nativeElement instead of Renderer2

The mistake	this.el.nativeElement.style.color = 'red' directly in a directive
Why developers make it	It's shorter, familiar from vanilla JS, and works correctly in a browser
How to avoid it	Always use Renderer2: this.renderer.setStyle(this.el.nativeElement, 'color', 'red')
Angular consequence	Breaks Angular Universal (SSR), Web Worker rendering, and test environments where nativeElement may not be a real DOM node

Mistake 2 — Forgetting to add the directive to the component's imports array

The mistake	Creating a standalone directive but not importing it in the consuming component's imports array
Why developers make it	In NgModule apps directives were in declarations. Muscle memory carries over.
How to avoid it	Treat standalone directives exactly like standalone components: they must be in imports: [MyDirective]
Angular consequence	Angular silently ignores the attribute — no error, no warning. The directive never instantiates.

Mistake 3 — Using @HostBinding to bind to innerHTML for content projection

The mistake	@HostBinding('innerHTML') content = 'Hello'
Why developers make it	Looks like a quick way to add content to the host element from a directive
How to avoid it	If you need to add a template, you need a component (with ng-content), not a directive
Angular consequence	Bypasses Angular's change detection and template compilation — event bindings inside the HTML don't work, XSS vulnerabilities introduced

Section 6 — Do's and Don'ts

DO	DON'T
DO: Use Renderer2 for all DOM manipulation inside directives	DON'T: Use ElementRef.nativeElement directly for style/attribute changes
DO: Use @HostBinding to bind to host element properties declaratively	DON'T: Manually set nativeElement.className or nativeElement.style in event handlers
DO: Use @HostListener('document:click') for global event listeners	DON'T: Add window.addEventListener manually — it leaks because Angular won't clean it up
DO: Add the directive to the component imports array (standalone)	DON'T: Assume directives auto-discover themselves
DO: Use @Input() with the same name as the selector for concise binding	DON'T: Create a separate @Input() config that requires verbose two-attribute syntax
DO: Use ngOnDestroy to clean up any subscriptions created inside the directive	DON'T: Forget that directives have their own lifecycle — subscriptions in directives leak

Section 7 — Debugging Tips

Bug 1 — Directive not working; element behaves as if directive was never applied

Symptom	No visual change, no console errors, @HostListener never fires
Cause	The directive is not imported in the consuming component's imports array
Fix	@Component({ standalone: true, imports: [HighlightDirective], ... })
Angular note	Enable strictTemplates in tsconfig.json — Angular will emit a compile-time error for unknown attributes.

Bug 2 — @HostListener('document:click') fires immediately on the same click that opened a dropdown

Symptom	User clicks a button to open a dropdown. The clickOutside event fires on the same click, immediately closing it.
Cause	The document:click listener fires for ALL clicks — including the one on the toggle button. The click bubbles up to the document.

Fix	Use stopPropagation on the toggle button: (click)="toggleDropdown(); $event.stopPropagation()"
Angular note	This is a DOM event timing issue. The stopPropagation approach prevents the toggle click from reaching the document listener.

Bug 3 — @HostBinding value not updating after @Input changes

Symptom	Directive applies correctly on first render, but when the bound @Input value changes, the @HostBinding doesn't update
Cause	The @HostBinding is set in ngOnInit and never updated — @Input changes fire ngOnChanges but the directive has no ngOnChanges handler
Fix	Implement OnChanges: ngOnChanges(): void { this.bg = this.appHighlight; }
Angular note	@HostBinding is reactive to its own property changes but doesn't automatically watch @Input properties.

Section 8 — Real-World Applications

Scenario 1 — Auto-focus directive for accessible modals

```
@Directive({ selector: '[appAutoFocus]', standalone: true })
export class AutoFocusDirective implements OnInit {
  @Input() appAutoFocus: string = ''; // CSS selector for which element to focus
  private el = inject(ElementRef);

  ngOnInit(): void {
    // setTimeout pushes focus call after Angular has finished rendering content
    setTimeout(() => {
      const target = this.appAutoFocus
        ? this.el.nativeElement.querySelector(this.appAutoFocus)
        : this.el.nativeElement;
      (target as HTMLElement)?.focus();
    });
  }
}

// Template usage:
// <div class="modal" appAutoFocus="input, button, [tabindex]">
```

Scenario 2 — Tooltip directive

```
@Directive({ selector: '[appTooltip]', standalone: true })
export class TooltipDirective implements OnDestroy {
  @Input() appTooltip = '';
  private el = inject(ElementRef);
  private renderer = inject(Renderer2);
  private tooltipElement: HTMLElement | null = null;

  @HostListener('mouseenter') onMouseEnter(): void { this.createTooltip(); }
  @HostListener('mouseleave') onMouseLeave(): void { this.removeTooltip(); }

  private createTooltip(): void {
    this.tooltipElement = this.renderer.createElement('div') as HTMLElement;
    this.renderer.addClass(this.tooltipElement, 'tooltip');
    const text = this.renderer.createText(this.appTooltip);
    this.renderer.appendChild(this.tooltipElement, text);
    const hostRect = this.el.nativeElement.getBoundingClientRect();
    this.renderer.setStyle(this.tooltipElement, 'position', 'fixed');
    this.renderer.setStyle(this.tooltipElement, 'top', `${hostRect.top - 36}px`);
    this.renderer.appendChild(document.body, this.tooltipElement);
  }

  private removeTooltip(): void {
    if (this.tooltipElement) {
      this.renderer.removeChild(document.body, this.tooltipElement);
      this.tooltipElement = null;
    }
  }

  ngOnDestroy(): void { this.removeTooltip(); }
}
```

Section 9 — Practice Exercises

Exercise 1 — Beginner: Build an appHighlight directive that accepts two @Input() values: appHighlight (hover color, defaults to 'yellow') and defaultColor (base color, defaults to 'transparent'). The directive should change the host element's backgroundColor on mouseenter and restore it on mouseleave using @HostBinding. Apply it to three different element types in a test component.

Exercise 2 — Intermediate: Build an appCopyToClipboard directive that reads its @Input() appCopyToClipboard: string value, listens to a click event, copies the text to the clipboard using navigator.clipboard.writeText(), and temporarily adds a CSS class 'copied' to the host element for 2 seconds after a successful copy. Use @HostBinding('class.copied') for the class toggle.

Exercise 3 — Advanced: Build a appPermission directive that accepts @Input() appPermission: string[] (array of required permissions). Inject a PermissionService that exposes permissions$: Observable<string[]>. The directive must reactively update the host element's disabled attribute and opacity style whenever permissions change. Use Renderer2 for all DOM manipulation. Clean up with takeUntilDestroyed.

Section 10 — Key Takeaways

- A directive attaches behaviour to an existing host element — it has no template of its own
- @HostBinding binds a directive property to a host element property — Angular keeps it in sync
- @HostListener subscribes to a host element event — Angular removes the listener on destroy
- @Input() on a directive works identically to @Input() on a component — same binding syntax
- Renderer2 is the platform-safe way to manipulate DOM — never use nativeElement directly
- Directives must be imported in every consuming component's imports array (standalone)

Decorator	What It Does	Example
@Directive({ selector })	Marks class as directive, sets CSS selector	selector: '[appHighlight]'
@HostBinding('prop')	Syncs directive property to host element property	@HostBinding('class.active') isActive = false
@HostListener('event')	Subscribes directive method to host DOM event	@HostListener('click') onClick()
@Input()	Receives value from template binding	@Input() color = 'yellow'
ElementRef	Reference to the raw host DOM element	inject(ElementRef)
Renderer2	Platform-safe DOM manipulation API	renderer.setStyle(el, 'color', 'red')

ONE-LINE MEMORY AID: A directive is a sticker on an existing element — @HostBinding changes how it looks, @HostListener reacts to what happens to it, and @Input configures what the sticker does.

Section 11 — Thought-Provoking Question + Answer

Question: When should you build a custom directive versus a custom component? Is there a clear architectural rule, or is it a judgment call?

Answer: There is a clear rule. A component owns its own template — it defines what is rendered inside it. A directive owns its own behaviour — it changes what an existing element does or looks like.

The test is: does this feature need its own HTML structure, or is it modifying an existing element?

Needs own HTML structure	-> Component
Modifies an existing element	-> Directive

A tooltip component wraps a trigger element with its own structure. A tooltip directive is applied to any existing element — the button already exists; the directive adds behaviour to it. Both work; the directive is more composable because it applies to any element type without restructuring the template.

Practical rule: If you catch yourself writing a component that has only <ng-content></ng-content> in its template and exists purely to attach behaviour — stop, delete it, and write a directive instead.

Section 12 — Related Topics to Learn Next

ES6/JS: MutationObserver — for directives that react to DOM changes inside the host element. IntersectionObserver — foundation of viewport-aware directives (lazy image loading, infinite scroll).

TypeScript: Generic classes — writing a directive that works across multiple typed host element types.

Angular-Specific:

- Topic 42 — Custom Pipes: The logical companion to directives — pipes transform data; directives add behaviour; together they cover template-layer reusability
- Topic 49 — Angular CDK: cdkDrag, cdkOverlay are sophisticated built-in directives
- exportAs on @Directive — allows template reference variables to reference directive instances

Section 13 — Study Plan Checkpoint

Directly depends on: Topic 17 (Decorators), Topic 25 (Lifecycle Hooks), Topic 26 (@Input/@Output), Topic 27 (DI), Topic 30 (Built-in Directives).

Directly unlocks: Topic 42 (Custom Pipes), Topic 45 (Performance Optimisation), Topic 49 (Angular CDK).

Production readiness: You are ready to build and ship custom directives — start with focused single-responsibility directives (auto-focus, click-outside, permission gating) before tackling complex ones.

Section 14 — Common Interview Questions

Q1 (Junior): What is the difference between a component and a directive in Angular?

Strong answer covers: Components have a template; directives do not; both are technically directives — @Component extends @Directive; attribute directives add behaviour; structural directives change DOM structure; when to choose each: own HTML structure -> component; modify existing element -> directive.

Weak answer misses: That @Component IS a directive with a template.

Q2 (Mid): What is Renderer2 and why should you use it instead of accessing nativeElement directly?

Strong answer covers: Renderer2 is Angular's platform-agnostic DOM abstraction; direct nativeElement access breaks in SSR, Web Workers, test environments; Renderer2 methods: setStyle, addClass, setAttribute, createElement, appendChild; Angular swaps the Renderer2 implementation based on rendering environment.

Weak answer misses: The SSR/platform-agnostic angle — just saying 'it's the Angular way' without explaining why.

Q3 (Mid-Senior): How would you build a directive that listens to clicks anywhere on the document to close a dropdown, while ensuring it doesn't close on the click that opened it?

Strong answer covers: @HostListener('document:click', ['$event.target']); ElementRef.nativeElement.contains(target) to distinguish inside vs outside; timing problem — open-click bubbles to document and triggers close; solution: stopPropagation on trigger, or setTimeout delay; @Output() clickOutside = new EventEmitter<void>(); cleanup automatic.

Weak answer misses: The timing/same-click problem.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Putting business logic inside a directive

```
// WRONG — directive doing HTTP calls
@Directive({ selector: '[appUserStatus]' })
export class UserStatusDirective implements OnInit {
  @Input() userId!: string;
  ngOnInit(): void {
    this.http.get<User>(`/api/users/${this.userId}`).subscribe(user => {
      if (user.isActive) this.renderer.addClass(this.el.nativeElement, 'active');
    });
  }
}

// CORRECT — directive receives resolved data via @Input
@Directive({ selector: '[appUserStatus]' })
export class UserStatusDirective implements OnChanges {
  @Input() isActive = false;
  @HostBinding('class.active') get activeClass() { return this.isActive; }
}
```

Anti-Pattern 2 — Using ElementRef.nativeElement for DOM manipulation

```
// WRONG — crashes in Angular Universal (SSR)
@HostListener('mouseenter')
onEnter(): void {
  this.el.nativeElement.style.backgroundColor = 'yellow';
  this.el.nativeElement.classList.add('highlighted');
}

// CORRECT — Renderer2 for all DOM operations
@HostListener('mouseenter')
onEnter(): void {
  this.renderer.setStyle(this.el.nativeElement, 'backgroundColor', 'yellow');
  this.renderer.addClass(this.el.nativeElement, 'highlighted');
}
```

Anti-Pattern 3 — Creating a directive when a component is the right tool

```
// WRONG — directive trying to build its own HTML structure via DOM manipulation
@Directive({ selector: '[appCard]' })
export class CardDirective implements OnInit {
  ngOnInit(): void {
    const header = this.renderer.createElement('div');
    // More DOM surgery... bypasses change detection entirely
  }
}

// CORRECT — component with content projection for reusable structure
@Component({
  selector: 'app-card',
  template: `
    <div class="card">
      <div class="card-header"><ng-content select="[header]" /></div>
      <div class="card-body"><ng-content /></div>
    </div>
  `,
})
export class CardComponent {}
```

Section 16 — Mental Model Summary

Core concept: A custom directive is a behaviour class that attaches to an existing host element via a CSS selector, reacting to its events with @HostListener and modifying its properties with @HostBinding — with no template of its own.

- 1. Directives have no template — they enhance an existing element, not replace it
- 2. Use @HostBinding to declaratively sync directive state to host element properties
- 3. Use @HostListener to react to host element events — Angular handles listener cleanup
- 4. Always use Renderer2 for DOM manipulation — never nativeElement directly
- 5. Import standalone directives in every consuming component's imports array
- 6. MOST IMPORTANT: If the feature needs its own HTML structure, build a component — not a directive

	Component
Has own template	Yes
@HostBinding	Yes
@HostListener	Yes
@Input() / @Output()	Yes

Lifecycle hooks	Yes
When to use	Needs own HTML structure
Content projection	Yes (ng-content)
